

IsoTrace: Explaining Transaction Isolation Failures with Dependency Graph Witnesses

An artifact report for bounded serializability analysis over explicit histories

IsoTrace contributors

May 2026

Abstract

IsoTrace is a bounded isolation-analysis workbench for explicit transaction histories. It answers a narrow systems question: given a history with read provenance and modeled predicate evidence, can the artifact reconstruct dependency edges, decide whether a serial order exists, and explain any failure as a concrete witness? The current implementation builds dependency graphs with write-write (**ww**), write-read (**wr**), read-write anti-dependency (**rw**), predicate-read/write anti-dependency (**prw**), and realtime (**rt**) edges. It then extracts deterministic cycle witnesses or, for clean histories, deterministic order witnesses. The artifact emits conservative semantic verdicts for supported cases: write skew, strict stale read, generic dependency cycle, valid serial history, aborted-write handling, and explicit predicate phantom-style failures. Evidence is preserved across fixture contracts, JSON schemas, report envelopes, command-line output, a browser workbench, UI smoke checks, and artifact validation. The claims here are intentionally bounded: IsoTrace does not connect to live databases, does not implement a general SQL parser, does not infer missing database snapshots or ranges, and does not claim full Elle compatibility, full Adya anomaly coverage, or database certification.

1 Research Question and Contributions

The artifact asks: given an explicit transaction history with read provenance and predicate evidence, can IsoTrace reconstruct dependency edges, identify whether a serial order exists, and explain the failure as a concrete witness?

This question is smaller than database certification and larger than a dashboard over traces. The hard object is the dependency graph. A history may look locally plausible: each read names a value, each write names a key, and each predicate read returns a row set. The isolation failure only becomes visible when the analyzer connects read-from facts, version order, realtime order, and modeled predicate membership changes into a graph. A directed cycle in that graph is the reason no serial order can satisfy all observed facts.

IsoTrace contributes four artifact mechanisms.

1. A runtime and schema-validated explicit history model for point reads, writes, timestamps, transaction status, and modeled predicate reads.
2. A deterministic dependency graph builder that constructs **ww**, **wr**, **rw**, **prw**, and **rt** edges without inventing facts absent from the input.
3. A verdict layer that maps graph evidence into bounded isolation diagnoses and exposes implicated transactions, proof edges, explanations, limitations, and inspection hints.
4. A reproducibility path: checked-in fixtures, fixture contracts, JSON report envelopes, CLI proof output, UI smoke, and an artifact checker that catches drift.

Claims and non-claims. IsoTrace claims that the current implementation reconstructs the supported edge classes from explicit histories and explains supported cycle shapes as bounded verdicts. It claims that the seven checked-in fixtures and their expected contracts are reproducible through the listed commands. It does not claim to observe real database executions, certify database implementations, infer omitted rows, parse general SQL, or implement the full anomaly taxonomies associated with Elle or Adya-style theory.

2 History Model

IsoTrace histories are local JSON artifacts. A transaction has an identifier, optional begin and commit times, optional status, and a list of operations. A point read records a key, value, and **from** transaction. The **from** field is not optional evidence: it states which committed transaction version the read observed. A write records a key and value. The v1 model accepts one write per key per transaction, rejects ambiguous read provenance, and rejects mixed explicit and missing commit times among non-initial committed transactions.

This design trades breadth for inspectability. IsoTrace does not infer read-from relationships from values alone. If two transactions wrote the same value, the history still must say which transaction a read observed. Likewise, T0 is treated as an initial seed transaction when present; non-initial committed transactions are ordered either by numeric commit time or by fixture order, but not by a mixture of both. These choices make the resulting graph reproducible rather than heuristic.

Predicate reads are explicit operations rather than parsed SQL queries. A predicate read names a table, a predicate object, and the returned rows. Supported predicates are leaf comparisons over a column, plus composite **all**, **any**, and **not** predicates. Relational write metadata may name a table, row id, mutation kind, prior row, post row, and fields. This metadata is used only when it is supplied and evaluable. Missing or unknown predicate membership does not produce a **prw** edge.

3 Dependency Graph and Verdict Engine

The graph builder creates one node per committed participating transaction. Aborted transactions do not participate in cycle evidence, although the verdict layer can report that an aborted write was ignored when that is the meaningful outcome. The supported edge kinds are:

- **ww**: version order between committed writes to the same key.
- **wr**: a committed transaction reads a version written by another committed transaction.
- **rw**: a transaction read an older version before another transaction wrote a later version to the same key.
- **prw**: a predicate read observed a row-membership state before another transaction changed membership of a modeled row.
- **rt**: strict-serializability realtime order, included only when strict mode is evaluated.

Serializability is evaluated from dependency cycles without treating realtime-only order as a serializability anomaly. Strict serializability includes realtime evidence when strict mode and timestamps are present. For acyclic graphs, IsoTrace reports a deterministic topological order witness. For cyclic graphs, it extracts a deterministic cycle witness and records the proof edge sequence. This witness is the central product surface: the CLI prints it, the JSON report contains it, and the workbench lets the user select the corresponding edge.

The verdict layer maps graph evidence to conservative labels. A two-transaction **rw/rw** cycle is reported as write skew. A cycle involving realtime and read/write evidence under strict mode is reported as strict stale read. A cycle containing **prw** is reported as an explicit predicate phantom-style dependency. Other supported dependency cycles fall back to a generic dependency-cycle label. Clean histories receive a valid serial history verdict with an order witness. Aborted-write examples can receive an aborted-write ignored verdict when ignored aborted state is the important evidence.

4 Predicate-Read Phantom Evidence

The predicate model exists to make one narrow class of phantom-style evidence inspectable without pretending to be a database snapshot engine. A **prw** edge is added when a committed predicate reader and a committed writer touch the same table, the writer supplies enough row evidence, and the modeled row's membership changes with respect to the reader's predicate. If membership is unknown, IsoTrace skips the edge and records bounded notes rather than inventing a missing row.

Figure 1 shows the implemented `phantom_predicate_cycle` fixture. The initial seed contains alice with `on_call=true` and bob with `on_call=false`. Transaction T1 predicate-reads doctors where `on_call=true`, returns alice, and updates bob to `on_call=true`. Transaction T2 predicate-reads doctors where `on_call=false`, returns

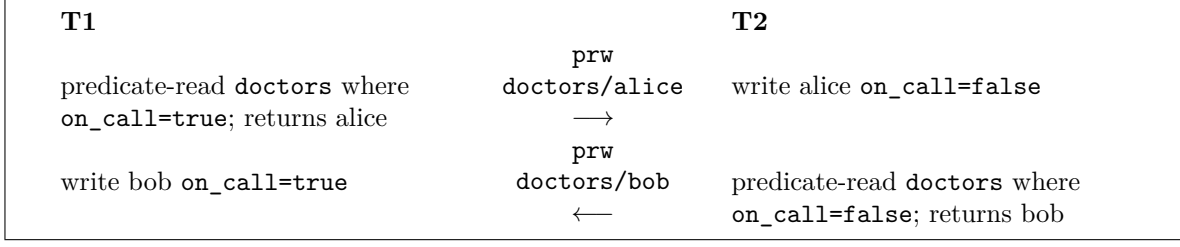


Figure 1: The implemented predicate phantom fixture. The proof depends on supplied predicate rows and row-update evidence. It is not SQL range inference and not a database snapshot reconstruction.

Fixture	Anomaly label	Edges	Witness	Status
aborted_ write_ ignored	aborted-write-ignored	–	order T0 -> T2	pass
composite_ predicate_ delete_ cycle	predicate-dependency-cycle	prw/prw	cycle 1	fail
serial_ stock_ decrement	valid-serial-history	–	order T0 -> T1 -> T2	pass
stale_ read_ strict	strict-stale-read	rt/rw	cycle 1	fail
strict_ serial_ handoff	valid-serial-history	–	order T0 -> T1 -> T2	pass
phantom_ predicate_ cycle	predicate-dependency-cycle	prw/prw	cycle 1	fail
write_ skew_ doctors	write-skew	rw/rw	cycle 1	fail

Table 1: Fixture evidence from `fixtures/manifest.json`. These are bounded expectations for checked-in synthetic fixtures, not empirical database results.

bob, and updates alice to `on_call=false`. Each transaction changes membership in the predicate result observed by the other. The resulting proof cycle is T1 -> T2 on `doctors/alice` and T2 -> T1 on `doctors/bob`.

For `prw` edges, IsoTrace records structured predicate proof rows. The proof contains table, row id, predicate, mutation, before-membership, after-membership, row evidence, and source labels such as returned rows, fields, row-before, row-after, delete, or none. In the CLI, this appears as compact before/after lines. In JSON reports and the UI, it remains structured so downstream tooling can audit the same fact without scraping prose.

5 Evaluation and Artifact Proof

The current artifact proof is a bounded regression and reproducibility surface, not a statistical performance study. The fixture manifest contains seven fixtures. Each fixture has an expected verdict contract, reproduction command, edge-kind expectation, cycle count or order witness, and status. The artifact checker validates fixture coverage, fixture contracts, examples, generated fixture reports, CLI JSON reports, fixture catalog output, and the CI workflow.

The verified product check currently reports 86 passing tests. These tests cover graph behavior, analyzer contracts, verdict stability, predicate evaluation and exclusion, report stability, SQL trace import constraints, CLI behavior, artifact validation, and UI smoke. The browser smoke verifies that the workbench loads, fixture selection works, custom JSON import works, invalid histories render validation errors, valid histories show order witnesses, and predicate-cycle fixtures expose selectable proof edges.

The benchmark command is intentionally treated as a smoke check. It constructs generated serial histories at increasing sizes and reports local wall-clock times for graph construction and cycle search. The paper does not use those times as performance claims. Their role is to keep a reproducible benchmark command alive and catch gross regressions in the product proof loop.

6 Workbench Surface

The browser workbench is subordinate to the analyzer. Its purpose is to expose the technical object: selected history, semantic verdict, dependency graph, edge table, cycle witness, selected-edge details, validation notes, and order witnesses for clean histories. It also accepts pasted custom history JSON and runs the same analyzer path as the CLI.

The most important workbench behavior is edge-centered inspection. A verdict proof edge can be selected; the graph, history rows, operation chips, edge table, and selected-edge panel all reflect that edge. For predicate edges, the selected-edge panel shows predicate row evidence, including before and after membership. This makes the hard part visible: the user can inspect not only that a cycle exists, but which modeled fact caused an edge to exist.

The command-line interface remains the stronger reproducibility surface. It can validate a history, print a human proof, emit a JSON report envelope, list fixture contracts, parse the constrained annotated SQL trace format, and fail with status 2 when `-fail-on-violation` finds an anomaly. The UI is useful for demo legibility; the CLI and report envelopes are the stable artifact record.

7 Limitations

The limitations are part of the artifact’s credibility.

- IsoTrace analyzes explicit histories. It does not run workloads against a database.
- There is no live database adapter and no certification of database behavior.
- The SQL trace importer is deliberately constrained annotated syntax, not a general SQL parser.
- Predicate phantom-style evidence is limited to supplied predicate objects, returned rows, and modeled insert/update/delete row evidence.
- IsoTrace does not infer missing rows, database snapshots, ranges, joins, SQL expressions, or isolation behavior not represented in the input.
- The verdict labels are conservative and do not claim full Elle compatibility, full Adya anomaly coverage, or complete anomaly taxonomy coverage.
- Fixtures are synthetic and deterministic. They are product proof fixtures, not production traces.
- Benchmark timings are local smoke measurements and are not hardware-independent performance results.

8 Reproducibility Notes

The public commands are the reproduction interface. `npm run check` runs typechecking, tests, artifact checks, production build, and benchmark smoke. `npm run smoke:ui` runs CLI proof checks and a headless browser smoke when a browser backend is available. `npm run demo`, `npm run demo:strict`, and `npm run demo:phantom` print the core write-skew, strict stale-read, and explicit predicate phantom examples. `npm run artifacts:check` validates fixture contracts and report schemas.

The strongest single demo command is:

```
npm run demo:phantom
```

It prints **Result:** VIOLATION, **Anomaly:** Explicit predicate phantom [predicate-dependency-cycle], implicated transactions T1, T2, proof edges e3 -> e4 (prw -> prw), and before/after predicate row evidence. This is the compact artifact loop: explicit history in, graph proof out, bounded non-claims preserved.